

LECTURE 21

BCNF Schema Design by Decomposition

CSC270 · Database Systems

COMSATS University Islamabad, Lahore Campus

What We'll Cover

- The formal BCNF Decomposition Algorithm, as a repeatable loop
- Picking which FD to split on when a relation has several violations
- Why every split must be checked for the lossless-join property
- A complete, multi-step worked example, iterated to completion
- Dependency preservation — and why BCNF can sacrifice it
- BCNF vs. 3NF synthesis, side by side

BCNF

split → test →
repeat

What BCNF Requires

- A relation R is in Boyce-Codd Normal Form if, for every non-trivial FD $X \rightarrow Y$ that holds over R , X is a superkey of R
- Every determinant must be able to determine ALL of R 's attributes — not just the ones on the right-hand side
- Test: compute X^+ under R 's full FD set; confirm $X^+ = \text{all attributes of } R$
- If some FD's determinant fails the test, R violates BCNF — this lecture's algorithm fixes it

BCNF TEST

$X \rightarrow Y$ over R

$X^+ = \text{all attrs of } R ?$

Fails the test?

R violates BCNF — run the decomposition algorithm.

The BCNF Decomposition Loop

1. Pick a relation R

From the current decomposition (initially, the original relation)

2. Test R for BCNF

For every FD $X \rightarrow Y$, check whether X^+ covers all of R's attributes

3. Violation found?

If every determinant is a superkey, R is done — set it aside

4. Split R on $X \rightarrow Y$

$R_1 = X \cup Y$ $R_2 = X \cup (R - Y)$

5. Replace R with R_1, R_2

Project R's FDs onto each; loop back to step 1 for both

loop while any relation still violates BCNF

Easy to Get Wrong

1

X appears in both halves

Not a mistake — it's exactly what makes the join lossless. X becomes the connecting column between R_1 and R_2 , playing the role a foreign key plays after the split.

2

Re-test the NEW relations

Don't declare victory after one split. A single split can leave one or both halves still violating BCNF if the original had more than one problem FD — exactly what happens next.

A Course Enrollment Relation

Enrollment (studentNo, courseNo, instructorNo, room)

studentNo	courseNo	instructorNo	room
S1001	CSC270	I01	LT-3
S1002	CSC270	I01	LT-3
S1001	CSC211	I02	LT-1
S1003	CSC270	I01	LT-3
S1003	CSC211	I02	LT-1

Business rules (functional dependencies)

FD1: studentNo, courseNo → instructorNo, room

an enrollment fixes who teaches it and where

FD2: courseNo → instructorNo

single-section policy: one instructor per course

FD3: instructorNo → room

each instructor teaches from one fixed room

STEP 1

Test the Original Relation

Candidate key: {studentNo, courseNo} — its closure already reaches all four attributes via FD1, and no smaller set does.

FD	Determinant	Superkey of Enrollment?	BCNF?
FD1	studentNo, courseNo	Yes — it IS the key	OK
FD2	courseNo	No — $\text{courseNo}^+ = \{\text{courseNo}, \text{instructorNo}, \text{room}\}$	Violation
FD3	instructorNo	No — $\text{instructorNo}^+ = \{\text{instructorNo}, \text{room}\}$	Violation

Enrollment violates BCNF — twice over. The algorithm says: pick ONE violation and split.

STEP 2

First Split, on courseNo → instructorNo

$X = \text{courseNo}, Y = \text{instructorNo} \rightarrow R_1 = X \cup Y \quad R_2 = X \cup (R - Y)$

CourseInstructor (courseNo, instructorNo)

courseNo	instructorNo
CSC270	I01
CSC211	I02

StudentCourseRoom (studentNo, courseNo, room)

studentNo	courseNo	room
S1001	CSC270	LT-3
S1002	CSC270	LT-3
S1001	CSC211	LT-1
S1003	CSC270	LT-3
S1003	CSC211	LT-1

CourseInstructor carries FD2; StudentCourseRoom still needs testing — it inherited FD3's consequence.

Test Both New Relations

CourseInstructor(courseNo, instructorNo)

- Only projected FD: $\text{courseNo} \rightarrow \text{instructorNo}$
- courseNo is a candidate key — its closure reaches both attributes
- Its determinant IS the key

BCNF — done

StudentCourseRoom(studentNo, courseNo, room)

- Key still $\{\text{studentNo}, \text{courseNo}\}$
- $\text{courseNo} \rightarrow \text{room}$ holds indirectly ($\text{courseNo} \rightarrow \text{instructorNo} \rightarrow \text{room}$)
- $\text{courseNo}^+ = \{\text{courseNo}, \text{room}\}$ — does NOT cover studentNo
- courseNo is not a superkey here

Violation — split again

STEP 4

Second Split, on courseNo → room

Splitting StudentCourseRoom. X = courseNo, Y = room

CourseRoom (courseNo, room)

courseNo	room
CSC270	LT-3
CSC211	LT-1

StudentCourse (studentNo, courseNo)

studentNo	courseNo
S1001	CSC270
S1002	CSC270
S1001	CSC211
S1003	CSC270
S1003	CSC211

StudentCourse is all-key (studentNo, courseNo) — no non-trivial FDs remain, so it is automatically in BCNF.

Final BCNF Check on Every Piece

Relation	Candidate Key	Every Determinant a Superkey?	BCNF?
CourseInstructor(courseNo, instructorNo)	courseNo	Yes	Pass
CourseRoom(courseNo, room)	courseNo	Yes	Pass
StudentCourse(studentNo, courseNo)	studentNo, courseNo	Yes — all-key relation	Pass

Three relations, not two

A direct consequence of Enrollment having had two independent violations, not one. The first split fixed FD2's violation but inherited FD3's problem into one half, which needed a second pass to resolve. This is exactly the “repeat” in “iterative BCNF decomposition.”

Checking the Lossless-Join Property

- A decomposition is lossless if natural-joining the pieces reconstructs exactly the original relation — no rows lost, no spurious rows gained
- Theorem: R decomposes losslessly into R_1, R_2 iff $R_1 \cap R_2 \rightarrow R_1$ or $R_1 \cap R_2 \rightarrow R_2$
- Here $R_1 \cap R_2 = X$, and $X \rightarrow Y$ was the FD we split on — so $X \rightarrow R_1$ holds by construction, every time

Rejoining the three relations — 5 rows out, 5 rows in

```
StudentCourse ⋈ CourseInstructor ⋈ CourseRoom  
(join on courseNo, then studentNo+courseNo)
```

studentNo	courseNo	instructorNo	room
S1001	CSC270	I01	LT-3
S1002	CSC270	I01	LT-3
S1001	CSC211	I02	LT-1
S1003	CSC270	I01	LT-3
S1003	CSC211	I02	LT-1

Dependency Preservation

- A decomposition is dependency-preserving if every original FD holds directly in one relation, or can be inferred by combining local FDs — without a join
- 3NF synthesis always preserves every original FD — BCNF decomposition sometimes cannot, even when fully lossless
- In our example: FD2 lives entirely in CourseInstructor. FD3 is only implied by combining CourseInstructor + CourseRoom — checking it now depends on a join

A real design decision, not a footnote

When BCNF loses dependency preservation: (1) enforce the lost constraint with a trigger, (2) settle for 3NF on that relation, or (3) redesign the business rule — there is no universal right answer.

3NF Synthesis vs. BCNF Decomposition

3NF Synthesis

Always lossless AND always dependency-preserving — the synthesis algorithm builds one relation per FD specifically to guarantee this.

Trade-off: may retain a small amount of redundancy that BCNF would remove.

BCNF Decomposition

Always lossless, but NOT guaranteed dependency-preserving.

Trade-off: removes all redundancy tied to a non-key determinant, at the possible cost of needing a join to re-check some constraint.

Testing Relations for BCNF — the Checklist

1

List every attribute of the relation and every FD projected onto it

2

Find every candidate key by computing closures (Lecture 20)

3

For every non-trivial FD $X \rightarrow Y$ in that set, compute X^+

4

If X^+ covers all attributes, that FD is fine; if any FD fails, split on it

5

Stop only when every relation in the decomposition passes with no exceptions

Key Takeaways

- The BCNF algorithm is a loop: while some relation violates BCNF, split on a violating FD $X \rightarrow Y$ into $R_1 = X \cup Y$ and $R_2 = X \cup (R - Y)$, then re-test both halves.
- A relation can have more than one BCNF violation — fixing one split can leave or inherit a second, exactly as Enrollment needed two passes.
- Every split by this rule is automatically lossless: $R_1 \cap R_2 = X$ and $X \rightarrow R_1$ holds by construction — no separate proof needed.
- Unlike 3NF synthesis, BCNF decomposition is NOT guaranteed to preserve every original FD — a genuine trade-off worth naming explicitly.
- The re-test-every-piece discipline, driven by closure computation, is the engine of the whole algorithm.